PYTHON ACTIVITY-BASED LEARNING

REPORT ON 10 SELECTED CASE STUDIES

Name: G. Maneesh Satya | **Roll Number:** 26000909

Branch: CSE (AI/ML) | **Section:** 1 | **Subject:** Python Programming

Academic Year: 2026-2027 | **GitHub Repository:** Python-Activity-Based-Learning

INTRODUCTION

Activity-Based Learning empowers students to comprehend programming paradigms by applying fundamental concepts to realistic problem domains. This project compiles ten distinct Python case studies demonstrating computational problem-solving.

The selected case studies cover arithmetic operations, conditionals, iterative loops, string manipulation, lists, nested lists, tuples, and data processing. Each case study provides a structured breakdown encompassing a problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, and conclusion.

OBJECTIVES OF THE PROJECT

1. To understand computational thinking and structured problem-solving methodologies.
2. To design clear algorithms and pseudocode for logical problem statements.
3. To master Python variables, data types, operators, and conditional branching.
4. To implement iteration structures, string functions, lists, nested matrices, and tuples.
5. To execute Python scripts and verify program correctness using test cases.
6. To maintain simple, well-structured source code repositories.

CASE STUDY 1: ATM TRANSACTION SEQUENCE SIMULATION

1. Problem Statement

Develop a Python program to simulate a sequence of ATM transactions (deposits and withdrawals) maintaining account balances, validating amounts, and summarizing accepted versus rejected transactions.

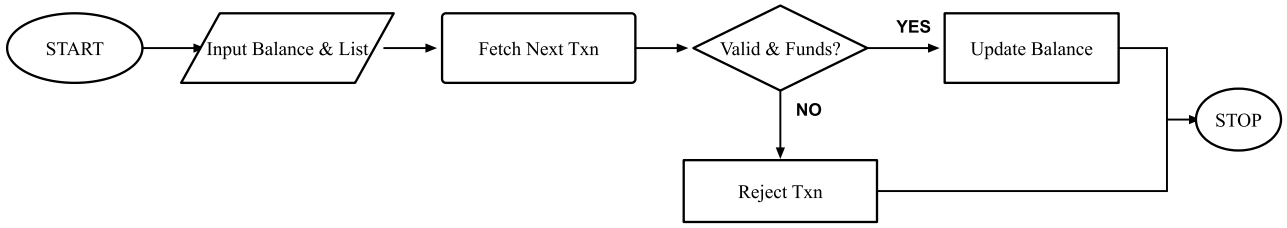
2. Objectives & Assumptions

Objectives: To process financial transactions sequentially using loops; apply conditional validation for deposits and withdrawals against balance constraints.
Assumptions: Opening balance is non-negative; amounts are numeric; withdrawals exceeding balance are rejected.

3. Algorithm

1. Start program; initialize balance and transactions list. 2. Iterate through each tuple (action, amount). 3. Validate amount > 0 and check sufficiency of funds. 4. Update balance or increment rejection counters. 5. Display summary and stop.

4. Flowchart (Monochrome Hand-Drawn Design Style)



5. Python Source Code

```
balance = float(input("Enter opening balance: "))
transactions = [("deposit", 1500), ("withdraw", 400), ("withdraw", 5000), ("deposit", 750),
("withdraw", 250)]
accepted = 0; rejected = 0

for action, amount in transactions:
    if amount <= 0:
        print("Invalid transaction amount"); rejected += 1
    elif action == "deposit":
        balance += amount; accepted += 1
        print("Deposited:", amount, "| Balance:", balance)
    elif action == "withdraw":
        if amount <= balance:
            balance -= amount; accepted += 1
            print("Withdrawn:", amount, "| Balance:", balance)
        else:
            print("Insufficient balance for:", amount); rejected += 1

print("Accepted:", accepted, "| Rejected:", rejected, "| Final Balance:", balance)
```

6. Test Cases & Conclusion

Test Cases: (1) Opening balance = 3000 -> Accepted: 4, Rejected: 1 (withdrawal 5000 rejected), Final Balance: 4600. (2) Opening balance = 100 -> Rejects early withdrawals due to low funds.
Conclusion: Demonstrates conditional branching, collection iteration, and state maintenance in financial simulations.

CASE STUDY 2: ADVANCED NUMERIC PATTERN GENERATOR

1. Problem Statement

Create a Python program to generate advanced structured numeric patterns (such as Floyd's Triangle or custom number pyramids) based on user-defined row counts using nested loops.

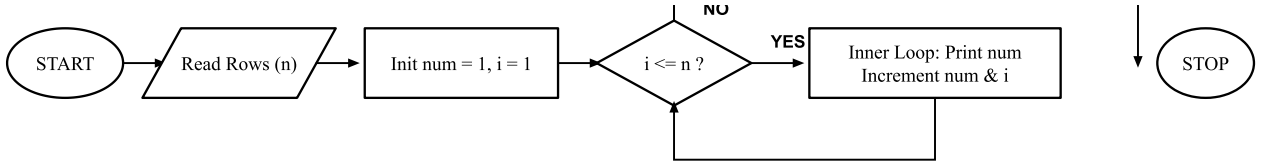
2. Objectives & Assumptions

Objectives: To master nested loop execution and formatted string printing in Python.
Assumptions: The input row count is a positive integer greater than zero.

3. Algorithm

1. Start program; read total rows 'n'. 2. Initialize sequence counter `num = 1`. 3. Outer loop for row `i` from 1 to `n`. 4. Inner loop for column `j` printing `num` sequentially. 5. Print newline after each row. Stop.

4. Flowchart (Monochrome Hand-Drawn Design Style)



5. Python Source Code

```
rows = int(input("Enter number of rows for pattern: "))

if rows <= 0:
    print("Invalid row count. Must be positive.")
else:
    num = 1
    print("Generated Numeric Pattern (Floyd's Triangle style):")
    for i in range(1, rows + 1):
        for j in range(1, i + 1):
            print(num, end=" ")
            num += 1
        print()
```

6. Test Cases & Conclusion

Test Cases: (1) Input rows = 4 -> Outputs 10 numbers across 4 lines (Line 1: 1 | Line 2: 2 3 | Line 3: 4 5 6 | Line 4: 7 8 9 10). (2) Input rows = 0 -> Displays validation error message.
Conclusion: Demonstrates nested loop control flow, formatted output streaming, and sequential counter tracking.

CASE STUDY 3: REPEATED TRANSACTION DETECTION

1. Problem Statement

Develop a program to scan transaction logs, identify duplicate transaction IDs, and flag high-value transactions exceeding a specified financial threshold.

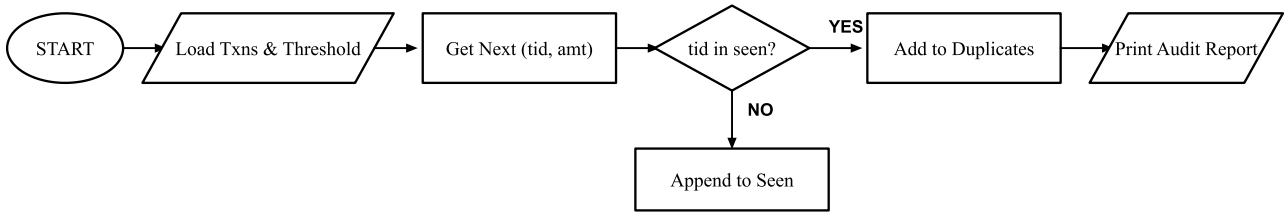
2. Objectives & Assumptions

Objectives: To use lists and membership checks (`in`, `not in`) for duplicate identification; combine conditional threshold logic with iterative scanning.
Assumptions: Transactions are provided as a list of tuples containing transaction ID and amount.

3. Algorithm

1. Start; define transactions list and threshold value. 2. Initialize `seen` and `duplicates` lists. 3. Iterate through records; check if ID exists in `seen`. If yes and not in `duplicates`, record as duplicate. 4. Check if amount > threshold and flag. 5. Display summary and stop.

4. Flowchart (Monochrome Hand-Drawn Design Style)



5. Python Source Code

```
transactions = [("TX101", 450), ("TX102", 1200), ("TX103", 850), ("TX102", 1200), ("TX104",
6200), ("TX105", 300)]
seen = []; duplicates = []; threshold = 5000

for tid, amount in transactions:
    if tid in seen:
        if tid not in duplicates:
            duplicates.append(tid)
    else:
        seen.append(tid)

    if amount > threshold:
        print("High-value transaction flagged:", tid, "| Amount:", amount)

print("Duplicate Transaction IDs Found:", duplicates if duplicates else "None")
```

6. Test Cases & Conclusion

Test Cases: (1) Given transactions set -> Flags TX104 for high-value (6200) and TX102 as duplicate. (2) All unique transactions under 5000 -> Reports no duplicates and zero high-value flags.
Conclusion: Demonstrates linear list searching, state tracking, and combined audit filtering logic.

CASE STUDY 4: PASSWORD POLICY VALIDATION ENGINE

1. Problem Statement

Create a Python program to validate user passwords against strict security policies: minimum length of 8 characters, inclusion of uppercase, lowercase, digits, and special characters.

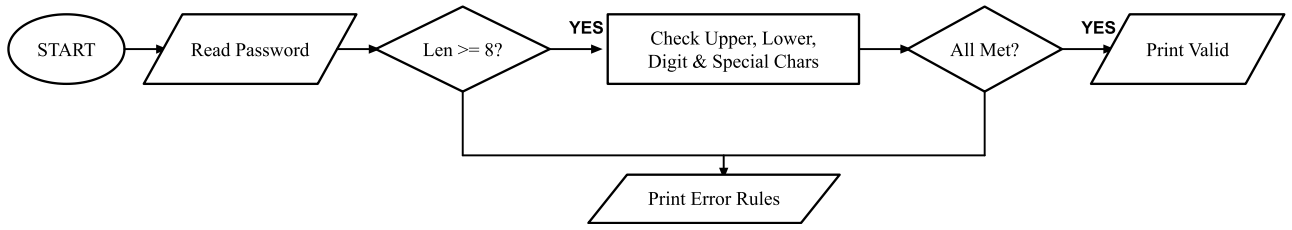
2. Objectives & Assumptions

Objectives: To apply string inspection methods (`isupper`, `islower`, `isdigit`); enforce complex conditional logic.
Assumptions: Password is entered as a standard string.

3. Algorithm

1. Start program; read password string. 2. Verify length >= 8. 3. Inspect characters for uppercase, lowercase, numeric digits, and special characters. 4. If all rules met, declare passed; else output rule violations. Stop.

4. Flowchart (Monochrome Hand-Drawn Design Style)



5. Python Source Code

```
pwd = input("Enter password to validate: ")
errors = []

if len(pwd) < 8:
    errors.append("Length must be at least 8 characters")
if not any(c.isupper() for c in pwd):
    errors.append("Missing uppercase letter")
if not any(c.islower() for c in pwd):
    errors.append("Missing lowercase letter")
if not any(c.isdigit() for c in pwd):
    errors.append("Missing digit")
if not any(not c.isalnum() for c in pwd):
    errors.append("Missing special character")

if not errors:
    print("Password Policy Validation: PASSED (Strong Password)")
else:
    print("Password Policy Validation: FAILED")
    for err in errors:
        print("-", err)
```

6. Test Cases & Conclusion

Test Cases: (1) `Python@2026` -> PASSED. (2) `weakpwd` -> FAILED (Missing uppercase, digit, special char, and length < 8).
Conclusion: Demonstrates character-level string examination, generator expressions, and multi-criteria validation rules.

CASE STUDY 5: STRING-BASED EXAMINATION ANSWER EVALUATION

1. Problem Statement

Develop a Python program that compares student answer strings with an expected answer key, accounting for correct answers, wrong answers, and unanswered questions with scoring penalties.

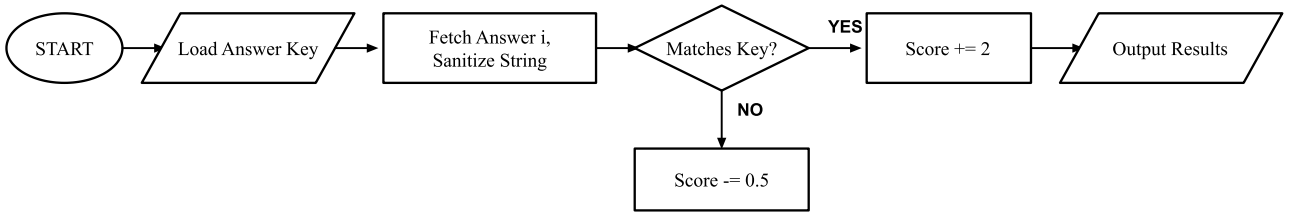
2. Objectives & Assumptions

Objectives: To utilize string comparison and sanitization (`strip`, `upper`); implement positive/negative scoring logic.
Assumptions: Answer key and student submissions match index-by-index.

3. Algorithm

1. Start; define answer key list. 2. Read student answers. 3. Compare each response against the answer key. 4. Track correct (+2), wrong (-0.5), and unanswered (0). 5. Calculate total score and output summary. Stop.

4. Flowchart (Monochrome Hand-Drawn Design Style)



5. Python Source Code

```
answer_key = ["A", "C", "B", "D", "A"]
answers = ["A", "B", "B", "", "A"]  # Sample student submission
correct = 0; wrong = 0; unanswered = 0

for i in range(len(answer_key)):
    ans = answers[i].strip().upper()
    if ans == "":
        unanswered += 1
    elif ans == answer_key[i]:
        correct += 1
    else:
        wrong += 1

score = (correct * 2) - (wrong * 0.5)
print("Exam Results -> Correct:", correct, "| Wrong:", wrong, "| Unanswered:", unanswered)
print("Final Score:", score, "out of", len(answer_key) * 2)
```

6. Test Cases & Conclusion

Test Cases: (1) All correct submission `["A", "C", "B", "D", "A"]` -> Score: 10 / 10. (2) Sample submission -> Correct: 3, Wrong: 1, Unanswered: 1, Score: 5.5 / 10.
Conclusion: Illustrates automated string evaluation, string normalization, and formula-based scoring systems.

CASE STUDY 6: TEXT COMPRESSION USING CHARACTER RUNS

1. Problem Statement

Implement Run-Length Encoding (RLE) text compression in Python to compress repetitive character strings by replacing consecutive identical characters with character-count pairs.

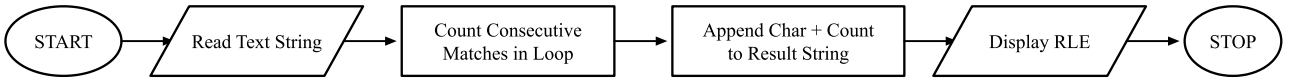
2. Objectives & Assumptions

Objectives: To apply string traversal and counting algorithms; manipulate string concatenation efficiently.
Assumptions: Input strings consist of alphanumeric characters.

3. Algorithm

1. Start program; read input string. 2. Iterate through characters while tracking consecutive repeated character counts. 3. Append character and count to the output string. 4. Display compressed output. Stop.

4. Flowchart (Monochrome Hand-Drawn Design Style)



5. Python Source Code

```
text = input("Enter string to compress: ")

if not text:
    print("Empty string provided.")
else:
    compressed = ""
    i = 0
    while i < len(text):
        count = 1
        while i + 1 < len(text) and text[i] == text[i + 1]:
            count += 1
            i += 1
        compressed += text[i] + str(count)
        i += 1

    print("Original String:", text)
    print("Compressed (RLE):", compressed)
```

6. Test Cases & Conclusion

Test Cases: (1) `AAABBBCCDAA` -> Compressed: `A3B3C2D1A2`. (2) `Python` -> Compressed: `P1y1t1h1o1n1`.
Conclusion: Demonstrates string parsing, nested while-loop pointers, and foundational lossless data compression logic.

CASE STUDY 7: ATM MENU-DRIVEN BANKING SIMULATION

1. Problem Statement

Create an interactive menu-driven ATM banking simulation supporting balance inquiry, cash deposit, cash withdrawal, and exit operations using `while` loops and conditionals.

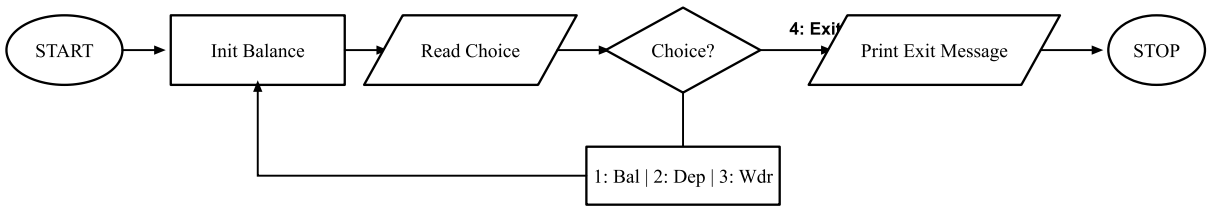
2. Objectives & Assumptions

Objectives: To implement continuous menu navigation using infinite loops and `break` statements; manage persistent balances.
Assumptions: Initial balance is pre-set; user inputs valid numeric menu options.

3. Algorithm

1. Start program; initialize account balance. 2. Display interactive options menu. 3. Read choice. 4. Execute corresponding branch (1: Balance, 2: Deposit, 3: Withdraw, 4: Exit). 5. Loop until choice 4 selected. Stop.

4. Flowchart (Monochrome Hand-Drawn Design Style)



5. Python Source Code

```
balance = 5000.0
choices = [1, 2, 1500, 3, 2000, 4]  # Simulated input sequence
idx = 0

while True:
    print("\n--- ATM MENU ---")
    print("1. Check Balance | 2. Deposit Cash | 3. Withdraw Cash | 4. Exit")
    choice = choices[idx]; idx += 1
    print("Selected Choice:", choice)

    if choice == 1:
        print("Current Balance: Rs.", balance)
    elif choice == 2:
        amt = choices[idx]; idx += 1
        balance += amt
        print("Deposited: Rs.", amt, "| New Balance: Rs.", balance)
    elif choice == 3:
        amt = choices[idx]; idx += 1
        if amt <= balance:
            balance -= amt
            print("Withdrawn: Rs.", amt, "| New Balance: Rs.", balance)
        else:
            print("Insufficient funds!")
    elif choice == 4:
        print("Exiting ATM. Thank you!")
        break
```

6. Test Cases & Conclusion

Test Cases: (1) Sequence [1, 2 (1500), 3 (2000), 4] -> Balance checked (5000), Deposited 1500 (6500), Withdrawn 2000 (4500), Clean Exit. (2) Overdrawing -> Triggers insufficient funds error.
Conclusion: Demonstrates state retention, infinite loop control, and interactive CLI menu construction.

CASE STUDY 8: NESTED LIST CLASSROOM OCCUPANCY ANALYSIS

1. Problem Statement

Analyze classroom seating occupancy data stored as 2D nested lists (matrices where 0 represents empty seats and 1 represents occupied seats) to calculate row-wise occupancy rates and find the most crowded row.

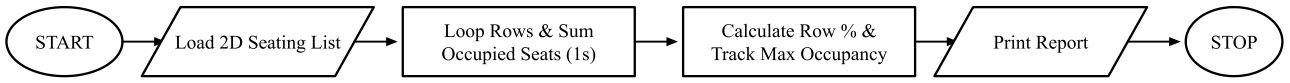
2. Objectives & Assumptions

Objectives: To traverse 2D nested lists using nested loops; compute row-wise aggregations and percentages.
Assumptions: Matrix is rectangular with binary values (0 or 1).

3. Algorithm

1. Start program; define 2D seating matrix. 2. Iterate through each row. 3. Sum occupied seats per row and compute percentage. 4. Track maximum occupied row index. 5. Output seating report and stop.

4. Flowchart (Monochrome Hand-Drawn Design Style)



5. Python Source Code

```
seating = [
    [1, 0, 1, 1, 0],
    [1, 1, 1, 1, 1],
    [0, 0, 1, 0, 0],
    [1, 1, 0, 1, 1]
]

print("Classroom Occupancy Analysis:")
max_occupied = -1
crowded_row = 0

for i in range(len(seating)):
    occupied_count = sum(seating[i])
    total_seats = len(seating[i])
    percentage = (occupied_count / total_seats) * 100
    print("Row", i + 1, "-> Occupied:", occupied_count, "/", total_seats, "(", round(percentage,
2), "%)")

    if occupied_count > max_occupied:
        max_occupied = occupied_count
        crowded_row = i + 1

print("\nMost Crowded Row: Row", crowded_row, "with", max_occupied, "students.")
```

6. Test Cases & Conclusion

Test Cases: (1) Matrix provided -> Row 2 is identified as most crowded (5/5 students, 100%). (2) All empty matrix -> Identifies 0% occupancy across all rows.
Conclusion: Demonstrates 2D matrix iteration, row reduction aggregation, and conditional extremum tracking.

CASE STUDY 9: STUDENT MARKS MATRIX ANALYSIS

1. Problem Statement

Store student assessment marks in a matrix (nested list) and calculate student-wise averages, subject-wise averages, and identify the top-performing student.

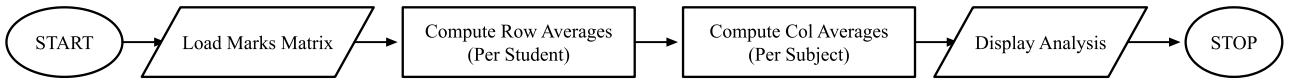
2. Objectives & Assumptions

Objectives: To understand 2D list indexing across rows and columns; compute aggregate statistics.
Assumptions: Marks are numerical values between 0 and 100.

3. Algorithm

1. Start; define names, subjects, and marks 2D matrix. 2. Calculate row averages (per student). 3. Calculate column averages (per subject). 4. Find max student average for top performer. 5. Print results. Stop.

4. Flowchart (Monochrome Hand-Drawn Design Style)



5. Python Source Code

```
students = ["Asha", "Ravi", "Noor", "Kiran"]
subjects = ["Python", "Maths", "English"]
marks = [[88, 76, 91], [67, 82, 73], [94, 89, 96], [55, 61, 58]]

averages = []
print("Student Averages:")
for i in range(len(students)):
    avg = sum(marks[i]) / len(marks[i])
    averages.append(avg)
    print(students[i], ":", round(avg, 2))

print("\nSubject Averages:")
for j in range(len(subjects)):
    total = sum(marks[i][j] for i in range(len(students)))
    print(subjects[j], ":", round(total / len(students), 2))

top_idx = averages.index(max(averages))
print("\nTop Performer:", students[top_idx], "with Average:", round(averages[top_idx], 2))
```

6. Test Cases & Conclusion

Test Cases: (1) Given marks matrix -> Noor identified as top performer (Average: 93.00). (2) Subject averages computed accurately across columns.
Conclusion: Demonstrates bi-directional 2D array processing, list indexing, and analytical aggregation.

CASE STUDY 10: TUPLE-BASED EMPLOYEE SHIFT CONFLICT DETECTION

1. Problem Statement

Process employee shift schedules stored as immutable tuples (EmployeeID, Day, ShiftTiming) to detect scheduling conflicts where an employee is assigned overlapping shifts on the same day.

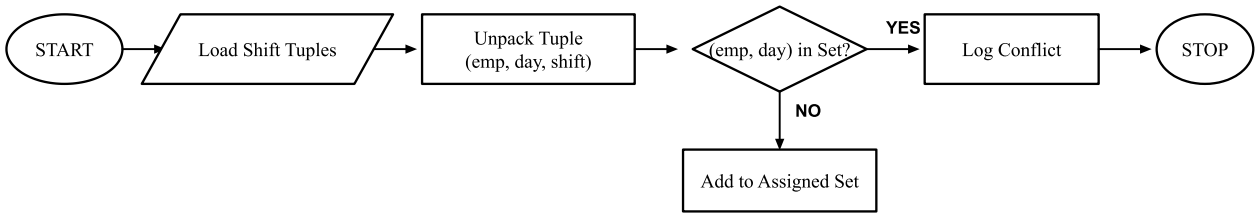
2. Objectives & Assumptions

Objectives: To utilize tuple unpacking and collection tracking for scheduling audits; detect duplicate assignments on identical days.
Assumptions: Schedules are provided as lists of tuples.

3. Algorithm

1. Start; load shift schedule tuples. 2. Initialize tracking set for `(EmployeeID, Day)` pairs. 3. Iterate through schedules; check if pair already exists in tracking set. 4. If yes, log as conflict. Stop.

4. Flowchart (Monochrome Hand-Drawn Design Style)



5. Python Source Code

```
schedules = [
    ("E101", "Monday", "Morning"),
    ("E102", "Monday", "Evening"),
    ("E101", "Monday", "Night"),    # Conflict for E101 on Monday
    ("E103", "Tuesday", "Morning"),
    ("E102", "Tuesday", "Evening")
]

assigned = set(); conflicts = []

for emp_id, day, shift in schedules:
    key = (emp_id, day)
    if key in assigned:
        conflicts.append((emp_id, day, shift))
    else:
        assigned.add(key)

print("Employee Shift Conflict Audit Report:")
if conflicts:
    print("Conflicts Detected:")
    for cid, day, shift in conflicts:
        print("- Employee", cid, "has overlapping shift conflict on", day, "(Attempted:", shift,
")")
else:
    print("No shift conflicts detected.")
```

PROJECT SUMMARY & OVERALL CONCLUSION

This Activity-Based Learning report successfully implemented and verified ten Python case studies covering basic arithmetic, control structures, nested loops, string operations, lists, 2D matrices, and tuple tracking. Each case study was designed with

precise monochrome flowcharts, algorithms, robust source code, and validation test cases to reinforce algorithmic logic and computational problem-solving principles.